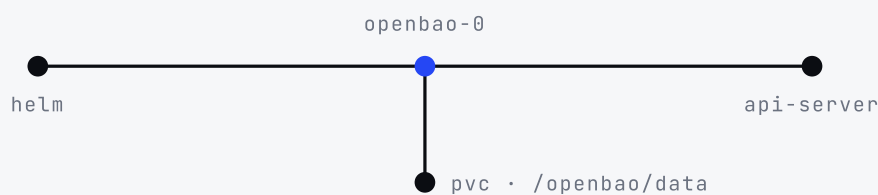


OPENBAO AUF KUBERNETES

Ein Secrets-Store im Cluster: Helm, Raft, init und unseal,
Kubernetes-Auth – und was danach noch fehlt.



OpenBao auf Kubernetes

Ein Secrets-Store im Cluster: Helm, Raft, init und unseal, Kubernetes-Auth – und was danach noch fehlt

Thomas Zachmann

17. September 2026

Inhalt

Worum es geht

- Warum von Hand, und warum ohne KI
- Ein Wort zu den Werten in diesem Leitfaden
- Für wen
- Die Bausteine
- Die Architektur in einem Bild

Teil I – Helm

- Das Chart
- Die Values
- Installieren
- Was helm upgrade später *nicht* tut

Teil II – init und unseal

- Warum das nicht automatisiert wird
- init
- unseal
- Was nach jedem Neustart passiert
- Der erste Login

Teil III – Kubernetes-Auth

- Wie es funktioniert
- Aktivieren und konfigurieren
- Eine Rolle für einen Consumer
- Prüfen, ohne ESO

Teil IV – Der Zugang von außen

- Der Weg
- Der bao-CLI von außen
- Die UI

Teil V – Der Ist-Zustand, ehrlich

Teil VI – Härtung: was als Nächstes kommt

1. Token-Datei aus dem Pod entfernen
2. Audit-Device
3. Admin-Policy und userpass
4. Den Ersatzzugang beweisen – nicht überspringen
5. Root-Token widerrufen
6. Snapshots
7. Konfiguration nach OpenTofu

Teil VII – Was schiefgehen kann

- Pod bleibt 0/1 Running
- Log voller service_registration ... 403

helm upgrade ändert nichts

403 permission denied bei allem

Kubernetes-Login schlägt fehl

OpenBao nimmt keine Anfragen mehr an, Log sagt audit

Nach Node-Wartung: versiegelt

Anhang A – Alle Kommandos

Anhang B – Glossar

Worum es geht

Ein Kubernetes-Cluster braucht einen Ort für Geheimnisse, der mehr kann als `kind: Secret` – ein Ort, der Zugriffe protokolliert, Credentials mit Ablaufdatum erzeugt und Workloads anhand ihrer Identität unterscheidet. Dieser Leitfaden baut genau das: **OpenBao im Cluster**, als StatefulSet mit Raft-Storage, per Helm installiert, manuell initialisiert und entsiegelt, mit Kubernetes-Auth als Login-Methode für Workloads.

Es ist der erste Teil einer Reihe. Der zweite, *Dynamische Datenbank-Credentials mit OpenBao*, setzt genau auf diesem Aufbau auf und lässt eine Anwendung kurzlebige PostgreSQL-Zugänge daraus beziehen. Der dritte beschreibt denselben OpenBao auf einer virtuellen Maschine – mit Ansible, OpenTofu und einem geübten Disaster Recovery. Viele Entscheidungen in diesem Leitfaden sind von dort übernommen, und wo der Cluster-Aufbau *hinter* dem VM-Aufbau zurückbleibt, steht das ausdrücklich dabei.

Alles hier wurde tatsächlich auf einem RKE2-Cluster im Homelab durchgeführt. Der Ist-Zustand am Ende ist ehrlich beschrieben – inklusive dessen, was noch nicht gemacht ist.

Warum von Hand, und warum ohne KI

In einem echten Setup wird ein Helm-Release nicht per Hand deployt, sondern per GitOps, und die OpenBao-Konfiguration nicht per CLI, sondern per OpenTofu. Trotzdem geht dieser Leitfaden jeden Schritt einmal von Hand – und bewusst ohne KI-Assistenz als Abkürzung. Wer ein Sprachmodell die Values schreiben lässt, bekommt ein laufendes OpenBao und keine Ahnung, warum `tlsDisable` gesetzt ist, was Raft von `file` unterscheidet oder warum der Pod nach jedem Neustart versiegelt ist.

Der Maßstab: Wer diesen Leitfaden durchgearbeitet hat, kann auf einem leeren Blatt zeichnen, wie ein Pod im Cluster an ein Secret aus OpenBao kommt – über welche Objekte, mit welchem Token, geprüft von wem. Wer das kann, kann auch das Helm-Release in Flux oder Argo einhängen und die Konfiguration nach OpenTofu übertragen.

Ein Wort zu den Werten in diesem Leitfaden

Der Cluster ist eine Entwicklungsumgebung: kein Zugang von außen, keine echten Daten. Hostnamen und interne Adressen sind durch Platzhalter ersetzt (`bao.example.internal`). Unseal-Keys und Root-Token erscheinen nirgends – sie stehen auch im Original nicht im Terminal-Log, weil `init` bewusst nie in einer Automatisierung oder einem Chat-Fenster läuft.

Für wen

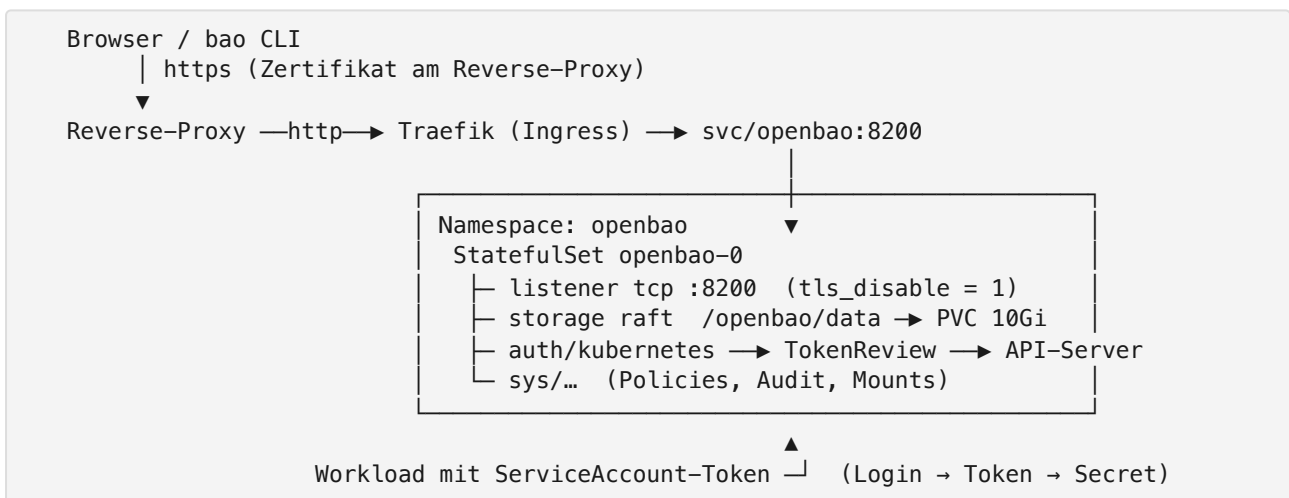
Für Leserinnen und Leser, die Kubernetes-Grundlagen kennen (StatefulSet, PVC, Ingress, ServiceAccount, Helm) und wissen, was ein Secrets-Manager ist. OpenBao ist ein Fork von HashiCorp Vault; alles hier gilt für beide, nur der CLI-Name (`bao`) und die Umgebungsvariablen (`BAO_ADDR`) unterscheiden sich.

Wer die Konzepte – Seal/Unseal, Auth-Methoden, Policies, Secrets Engines, Leases – von Grund auf verstehen will, mit Labs, die auf dem Laptop laufen, findet das in meinem Buch **Vault in Practice** (Leanpub, leanpub.com/vault-in-practice). Dieser Leitfaden setzt diese Konzepte voraus und zeigt, wie sie im Cluster zusammenkommen.

Die Bausteine

Baustein	Version	Rolle
RKE2 Kubernetes	v1.36.4+rke2r1	3 Control-Plane-Nodes, 3 Worker, Traefik als Ingress, Longhorn als Storage
OpenBao Helm-Chart openbao/openbao	Chart 0.29.4, App 2.6.2	Das Deployment
Helm	3.16	Installiert das Chart
bao CLI	2.6.2	Im Pod enthalten; lokal optional

Die Architektur in einem Bild



Drei Dinge, die man aus dem Bild mitnehmen sollte:

1. **TLS endet vor dem Cluster.** Der Listener spricht Klartext-HTTP. Das ist eine bewusste Vereinfachung für ein Homelab hinter einem Reverse-Proxy – und die erste Stelle, die man in einer produktiven Umgebung ändert.
2. **Der Zustand liegt in einem PVC.** Raft schreibt nach /openbao/data. Verliert man das Volume, verliert man alles – Snapshots sind darum kein Komfort, sondern Pflicht.
3. **Workloads melden sich mit dem an, was sie ohnehin haben:** ihrem ServiceAccount-Token. OpenBao fragt den API-Server, ob das Token echt ist. Kein zweites Geheimnis muss verteilt werden.

Teil I – Helm

Das Chart

OpenBao pflegt ein eigenes Chart, abgeleitet vom Vault-Chart. Es kann drei Betriebsarten: dev (In-Memory, unversiegelt – nur zum Spielen), standalone (ein Pod, persistentes Volume) und ha (mehrere Pods mit Raft-Quorum). Für ein Homelab mit einem physischen Host ist standalone die richtige Wahl – drei Pods auf demselben Host schützen nicht gegen den dominanten Ausfall (Host weg) und verdreifachen den Aufwand beim manuellen Entsiegeln.

```
helm repo add openbao https://openbao.github.io/openbao-helm
helm repo update
helm search repo openbao/openbao --versions | head -3
```

Die Values

Die vollständige Datei (k8s/values.yaml) hat 70 Zeilen; hier die Teile mit Begründung.

```
global:
  tlsDisable: true
```

TLS wird am Reverse-Proxy terminiert. Das Chart nutzt diesen Schalter an mehreren Stellen: Es setzt BA0_ADDR im Pod auf http://, konfiguriert die Probes ohne TLS und lässt den tls-Block im Ingress weg. Setzt man nur den Listener auf tls_disable = 1, aber nicht global.tlsDisable, gehen die Readiness-Probes ins Leere.

```
injector:
  enabled: false
```

Der Sidecar-Injector (vault-k8s) schreibt Secrets als Dateien in Pods. Für den geplanten Weg – External Secrets Operator – wird er nicht gebraucht. Jeder Controller, der nicht läuft, ist einer weniger, der Rechte im Cluster hat.

```
server:
  standalone:
    enabled: true
    config: |
      ui = true

      listener "tcp" {
        address      = "[::]:8200"
        cluster_address = "[::]:8201"
        tls_disable   = 1
      }

      storage "raft" {
        path = "/openbao/data"
      }
```

Der wichtigste Eingriff ist storage "raft". Der Chart-Default für standalone ist storage "file" – ein Backend, das **keine Snapshots kann**. `bao operator raft snapshot save` existiert nur mit Raft. Wer den Default lässt, hat keinen Weg, ein konsistentes Backup zu ziehen, außer das PVC zu kopieren – und das ist bei laufendem Prozess nicht konsistent.

`api_addr` und `cluster_addr` fehlen bewusst: Das Chart setzt `BA0_API_ADDR` und `BA0_CLUSTER_ADDR` als Umgebungsvariablen auf dem StatefulSet, auch im Standalone-Modus, und OpenBao liest sie beim Start.

`service_registration "kubernetes"` stand im ursprünglichen Setup ebenfalls in diesem Block – und ist in `k8s/values.yaml` bewusst **nicht** mehr enthalten. Die Registrierung soll den Zustand (`active`, `sealed`, ...) als Labels an den eigenen Pod schreiben, damit ein HA-Service nur den aktiven Knoten selektiert. Im Standalone-Modus rendert das Chart aber weder die Role noch das RoleBinding dafür, und OpenBao protokolliert dann alle fünf Sekunden:

```
[WARN] service_registration.kubernetes: unable to set initial state due; will retry:
err="GET https://10.43.0.1:443/api/v1/namespaces/openbao/pods/openbao-0 ...
resp statusCode: 403"
```

22 Stunden lang, bevor es jemandem auffiel. Die Services selektieren im Standalone-Modus ohnehin auf `component: server` mit `publishNotReadyAddresses: true` – die Labels würden nichts ändern. Also weglassen, oder eine Role mit `get/patch` auf pods ergänzen, wenn man die Labels für Monitoring will.

```
dataStorage:
  enabled: true
  size: 10Gi
  storageClass: longhorn

persistentVolumeClaimRetentionPolicy:
  whenDeleted: Retain
  whenScaled: Retain
```

Beides explizit, obwohl es die Defaults wären. Der Grund ist derselbe wie bei `ingressClassName`: Ein geänderter Cluster-Default darf die Raft-Daten nicht stillschweigend woanders hinlegen oder bei `helm uninstall` mitnehmen.

```
ingress:
  enabled: true
  ingressClassName: traefik
  hosts:
    - host: bao.example.internal
      paths: []

ui:
  enabled: true
  serviceType: ClusterIP
```

Kein `tls`-Block, kein `cert-manager`. Der Reverse-Proxy vor dem Cluster hat das Zertifikat und leitet per HTTP an Traefik weiter.

Installieren

```
helm upgrade --install openbao openbao/openbao \
  --version 0.29.4 \
  --namespace openbao --create-namespace \
  --values k8s/values.yaml
```

Ohne --wait. Die Readiness-Probe des Charts ist `bao status`, und die schlägt fehl, solange OpenBao versiegelt ist. Auf einer frischen Installation ist der Pod erst nach `init + unseal` *Ready* – `--wait` würde nur in den Timeout laufen.

Vorher prüfen, ohne etwas zu ändern:

```
helm upgrade --install openbao openbao/openbao --version 0.29.4 \
  --namespace openbao --values k8s/values.yaml --dry-run=server
```

Danach:

```
$ kubectl get sts,svc,ingress,pvc -n openbao
```

NAME	READY	AGE
statefulset.apps/openbao	0/1	1m

NAME	TYPE	CLUSTER-IP	PORT(S)
service/openbao	ClusterIP	10.43.179.240	8200/TCP,8201/TCP
service/openbao-internal	ClusterIP	None	8200/TCP,8201/TCP
service/openbao-ui	ClusterIP	10.43.158.186	8200/TCP

NAME	CLASS	HOSTS	PORTS
ingress.networking.k8s.io/openbao	traefik	bao.example.internal	80

NAME	STATUS	CAPACITY	STORAGECLASS
persistentvolumeclaim/data-openbao-0	Bound	10Gi	longhorn

READY 0/1 ist hier richtig. Der Pod läuft, OpenBao antwortet, aber es ist nicht initialisiert – und das Chart nennt das zu Recht „nicht bereit“.

Was helm upgrade später *nicht* tut

Das Chart setzt `updateStrategyType: OnDelete`. Eine geänderte `values.yaml` wird zu einer neuen ConfigMap, aber der Pod wird **nicht** neu gestartet. Erst `kubectl -n openbao delete pod openbao-0` übernimmt die Änderung – und danach ist OpenBao versiegelt. Das ist derselbe Kompromiss wie bei einem Konfigurations-Reload auf einer VM: Ein Neustart kostet ein Unseal, also passiert er nur absichtlich.

Teil II – init und unseal

Warum das nicht automatisiert wird

`bao operator init` erzeugt die Unseal-Keys und den Root-Token. Diese Ausgabe erscheint **genau einmal**. Läuft `init` in einer Pipeline, in Ansible oder in einem KI-Chat, steht das Key-Material in Logs, in Task-Ausgaben oder in einem Kontextfenster. Deshalb: ein normales Terminal, eine Shell im Pod, Ausgabe sofort in den Passwort-Manager, dann Terminal schließen.

init

```
kubectl exec -it -n openbao openbao-0 -- sh
/ $ bao operator init -key-shares=5 -key-threshold=3
```

Shamir 5 von 3: fünf Schlüssel, drei reichen zum Entsiegeln. Bei einem Homelab, in dem eine Person alle fünf hält, ist das kein Sicherheitsgewinn gegenüber 1/1 – aber es ist das Format, das ein späterer Wechsel zu mehreren Verwahrern erwartet, und es kostet nichts.

Die Ausgabe enthält Unseal Key 1 bis 5 und Initial Root Token. Alles in den Passwort-Manager. **Nicht** in dieses Repo, **nicht** auf das NAS, auf dem später die Snapshots liegen – die Snapshots sind mit genau diesen Keys verschlüsselt, und wer beides an einem Ort hat, hat beides verloren, wenn der Ort weg ist.

unseal

```
/ $ bao operator unseal      # Key eingeben, ohne Echo
/ $ bao operator unseal
/ $ bao operator unseal
/ $ bao status
```

Key	Value
Seal Type	shamir
Initialized	true
Sealed	false
Total Shares	5
Threshold	3
Version	2.6.2
Storage Type	raft
HA Enabled	true
HA Cluster	https://openbao-0.openbao-internal:8201
HA Mode	active

`HA Enabled true` bei einem einzelnen Pod überrascht – es ist eine Eigenschaft von Raft, nicht der Betriebsart. Der Pod ist der aktive (und einzige) Knoten.

Ohne Argument fragt `bao operator unseal` den Key interaktiv ab und liest ihn ohne Echo. Ein Key als Argument stünde in der Shell-History und wäre über `/proc` für andere Prozesse sichtbar.

Nach dem Unseal wechselt der Pod auf `READY 1/1`, und `service_registration` setzt `openbao-sealed: "false"` als Label – ab jetzt hat der Service ein Backend.

Was nach jedem Neustart passiert

Der Pod stirbt, wird verschoben, das Node wird gepatcht: OpenBao startet **versiegelt**. Das ist der bewusst akzeptierte Preis der Shamir-Entscheidung. Solange nichts davon abhängt, ist es nur lästig. Sobald der External Secrets Operator daran hängt (Nº 2), ist jeder Neustart ein Ausfall für alles, was Secrets erneuern muss. An diesem Punkt gehört Auto-Unseal (Transit-Engine auf dem VM-OpenBao, oder ein Cloud-KMS) auf die Liste.

Der erste Login

```
/ $ bao login
Token (will be hidden):
```

Der Root-Token. Ab hier ist die Shell `root` in OpenBao – und `bao login` schreibt das Token nach `~/.vault-token` im Container. Dazu gleich mehr in Teil V; erst einmal ist das der Zustand, in dem man die Konfiguration vornimmt.

Teil III – Kubernetes-Auth

Wie es funktioniert

Ein Pod hat ein ServiceAccount-Token (projected, mit Ablaufdatum). Der Pod schickt es an `auth/kubernetes/login`. OpenBao reicht es an den API-Server weiter (TokenReview), bekommt Namespace und Name des ServiceAccounts zurück, vergleicht mit der Rolle und stellt einen OpenBao-Token mit den Policies der Rolle aus.

Der Trick: Es muss kein Geheimnis verteilt werden. Der Pod hat sein Token ohnehin, und OpenBao hat sein eigenes Pod-Token und die Cluster-CA, um den TokenReview zu stellen.

Aktivieren und konfigurieren

```
/ $ bao auth enable kubernetes
/ $ bao write auth/kubernetes/config \
    kubernetes_host="https://kubernetes.default.svc:443"
```

Mehr nicht. Weil OpenBao selbst im Cluster läuft, nimmt es für den TokenReview sein eigenes ServiceAccount-Token und die CA aus `/var/run/secrets/kubernetes.io/serviceaccount/`. Die Berechtigung dafür bringt das Chart mit: Es rendert ein ClusterRoleBinding des ServiceAccounts `openbao` auf die eingebaute ClusterRole `system:auth-delegator` – das ist exakt das Recht, TokenReviews zu stellen, und sonst nichts. Läuft OpenBao **außerhalb** des Clusters, müssen `kubernetes_ca_cert` und `token_reviewer_jwt` explizit gesetzt werden – und das Reviewer-Token braucht die ClusterRole `system:auth-delegator`.

Eine Rolle für einen Consumer

Eine Rolle bindet einen ServiceAccount an Policies:

```
/ $ bao write auth/kubernetes/role/eso-demo \
    bound_service_account_names=demo \
    bound_service_account_namespaces=demo \
    token_policies=eso-demo \
    token_ttl=1h
```

```
/ $ bao read auth/kubernetes/role/eso-demo
Key                               Value
---                               -
alias_name_source                 serviceaccount_uid
bound_service_account_names       [demo]
bound_service_account_namespaces [demo]
token_policies                    [eso-demo]
token_ttl                         1h
```

alias_name_source: serviceaccount_uid bedeutet: Die Identität in OpenBao hängt an der UID des ServiceAccounts. Wird der ServiceAccount gelöscht und neu angelegt, ist es aus OpenBaos Sicht eine neue Identität – der alte Alias bleibt als Leiche im Identity-Store, bis man ihn entfernt.

Was die Policy eso-demo erlaubt, ist Sache des Consumers – in N° 2 sind das KV-Pfade und database/creds/*. Für diesen Leitfaden reicht: Die Rolle sagt *wer* darf sich anmelden und *welche* Policy er bekommt.

Prüfen, ohne ESO

Der schnellste Test ist ein Pod mit dem ServiceAccount, der den Login von Hand macht:

```
kubectl create namespace demo
kubectl create serviceaccount demo -n demo
kubectl run -n demo --rm -it --restart=Never \
  --overrides='{"spec":{"serviceAccountName":"demo"}}' \
  --image=curlimages/curl:8.11.1 probe -- sh
```

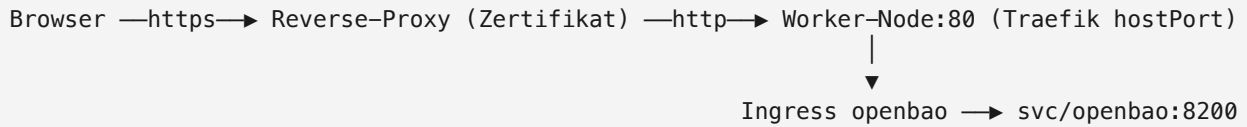
Im Pod:

```
JWT=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
curl -s http://openbao.openbao.svc:8200/v1/auth/kubernetes/login \
  -d '{"role":"eso-demo","jwt":"'$JWT'"}' | head -c 400
```

Ein client_token in der Antwort heißt: Auth-Methode, TokenReview und Rolle funktionieren. Eine 403 permission denied heißt fast immer: Namespace oder Name des ServiceAccounts passen nicht zur Rolle – nicht, dass die Policy etwas verbietet, denn die kommt erst nach dem Login ins Spiel.

Teil IV – Der Zugang von außen

Der Weg



Traefik läuft in RKE2 als DaemonSet mit hostPort 80/443 – aber nur auf den Workern, weil die Control-Plane-Nodes getaintet sind und das DaemonSet die Taints nicht toleriert. Das Ziel des Reverse-Proxys ist darum die IP eines Workers. Fällt dieser Worker aus, ist die UI nicht erreichbar, obwohl OpenBao weiterläuft.

Eine floatende VIP (MetalLB) würde das lösen. Sie wurde bewusst nicht installiert: Die Verfügbarkeit der UI ist in einem Homelab keinen weiteren Controller wert – die Workloads im Cluster erreichen OpenBao über svc/openbao, und das ist von keinem Worker abhängig.

Der bao-CLI von außen

```
export BAO_ADDR=https://bao.example.internal
bao status
bao login    # Token
```

Zwei Fallen, die beim Debuggen Zeit kosten:

- Ein Terminal mit BAO_ADDR auf den Ingress, aber ohne Token: **jeder** Aufruf antwortet 403 permission denied – auch `bao token capabilities`. Das sieht aus wie ein Policy-Problem und ist ein fehlendes Login. `bao token lookup` zuerst.
- `bao login` schreibt nach `~/.vault-token` – denselben Pfad, den die Vault-CLI benutzt. Wer parallel ein Vault oder einen zweiten OpenBao betreibt, überschreibt beim Login des einen das Token des anderen. Ein rätselhaftes 403 ist dann meist ein Token vom anderen System.

Die UI

`https://bao.example.internal/ui/` – Login mit dem Root-Token oder, nach der Härtung, per Username. Die UI ist praktisch für den Überblick über Mounts, Policies und Leases; für alles Reproduzierbare ist die CLI besser, weil ihre Aufrufe in ein Skript oder nach OpenTofu wandern können.

Teil V – Der Ist-Zustand, ehrlich

Das ist der Punkt, an dem dieser Leitfaden hinter N° 3 zurückbleibt. Auf der VM ist die Bootstrap-Sequenz vollständig durchgeführt und verifiziert. Im Cluster ist sie es **nicht**. So sieht es aus:

```
/ $ bao audit list
No audit devices are enabled.

/ $ bao auth list
Path          Type          Description
----          -
kubernetes/   kubernetes    n/a
token/        token         token based credentials

/ $ bao policy list
default
eso-demo
root

/ $ bao token lookup | grep -E 'display_name|policies'
display_name   root
policies       [root]

/ $ ls -la ~/.vault-token
-rw----- 1 openbao openbao /home/openbao/.vault-token
```

Vier Befunde:

- 1. Kein Audit-Device.** Niemand kann beantworten, wer wann welchen Pfad gelesen hat. Bei einem Secrets-Store ist das der Hauptgrund, ihn überhaupt zu betreiben.
- 2. Der Root-Token ist der einzige administrative Zugang.** Es gibt keinen userpass, keine Admin-Policy. Der Root-Token läuft nie ab und kann nicht eingeschränkt werden.
- 3. Der Root-Token liegt als Datei im Pod.** `bao login` hat ihn nach `/home/openbao/.vault-token` geschrieben. Jeder mit `kubectl exec` auf den Pod – also jeder mit ausreichenden RBAC-Rechten im Namespace `openbao` – ist damit root in OpenBao. Die Datei lebt im Container-Dateisystem und verschwindet beim Neustart; bis dahin ist sie da.
- 4. Keine Snapshots.** Das PVC ist die einzige Kopie der Raft-Daten. Longhorn repliziert das Volume zwar über Nodes, aber das schützt gegen einen Plattenausfall, nicht gegen ein versehentliches `helm uninstall` mit gelöschtem PVC, ein kaputtes Upgrade oder einen Fehlgriff mit `bao delete`.

Das ist der Zustand, in dem N° 2 gebaut wurde. Für eine Demo tragbar, für alles andere nicht. Teil VI beschreibt, was zu tun ist – die Kommandos sind aus dem VM-Runbook (N° 3) übernommen, wo sie geprobt sind.

Teil VI – Härtung: was als Nächstes kommt

Die Reihenfolge ist nicht beliebig. Audit zuerst, damit alles Folgende protokolliert ist. Root-Token zuletzt, und erst, wenn der Ersatzzugang **bewiesen** ist.

1. Token-Datei aus dem Pod entfernen

Sofort, unabhängig vom Rest:

```
/ $ rm ~/.vault-token
```

Für die weitere Arbeit das Token nur noch in der Umgebungsvariable der laufenden Shell halten:

```
/ $ read -rs BAO_TOKEN; export BAO_TOKEN
```

`read -rs` liest ohne Echo und ohne History-Eintrag. Beim Schließen der Shell ist das Token weg.

2. Audit-Device

```
/ $ bao audit enable file file_path=/openbao/audit/audit.log
```

`/openbao/audit` ist der Pfad, den das Chart mit `server.auditStorage.enabled: true` als eigenes PVC bereitstellt – ohne das Volume liegt das Log im Container und ist nach dem Neustart weg. Also erst die Values ergänzen:

```
server:
  auditStorage:
    enabled: true
    size: 2Gi
    storageClass: longhorn
```

Danach `helm upgrade`, Pod löschen, `unseal`, dann `audit enable`.

Zwei Dinge, die man wissen muss: Kann OpenBao nicht ins Audit-Log schreiben, **verweigert es Anfragen**. Ein volles Volume nimmt den Secrets-Store vom Netz. Das ist ein Sicherheitsfeature und ohne Rotation eine Zeitbombe – auf der VM übernimmt `logrotate` das; im Cluster braucht es einen Sidecar oder einen zweiten Audit-Device (`syslog`, `socket` zu einem Log-Collector), der zur primären Senke wird. Und: Ab jetzt zeigen alle Einträge die IP von Traefik statt des echten Clients, solange `x_forwarded_for_authorized_addrs` im Listener nicht gesetzt ist.

3. Admin-Policy und userpass

Auf der VM kommt das aus OpenTofu. Im Cluster ist der passende Weg derselbe – ein zweiter Tofu-Workspace oder ein zweites Verzeichnis, gegen `https://bao.example.internal`. Bis das steht, die CLI-Fassung:

```

/ $ bao policy write admin - <<'EOF'
path "sys/health" { capabilities = ["read", "sudo"] }
path "sys/capabilities-self" { capabilities = ["update"] }
path "sys/mounts" { capabilities = ["read", "list"] }
path "sys/mounts/*" { capabilities = ["create", "read", "update", "delete",
    "list", "sudo"] }
path "sys/auth" { capabilities = ["read", "list"] }
path "sys/auth/*" { capabilities = ["create", "read", "update", "delete",
    "sudo"] }
path "sys/policies/acl" { capabilities = ["list"] }
path "sys/policies/acl/*" { capabilities = ["create", "read", "update", "delete",
    "list"] }
path "sys/audit" { capabilities = ["read", "list", "sudo"] }
path "sys/audit/*" { capabilities = ["create", "read", "update", "delete",
    "sudo"] }
path "sys/leases/*" { capabilities = ["create", "read", "update", "delete",
    "list", "sudo"] }
path "sys/storage/raft/snapshot" { capabilities = ["read"] }

# Break glass: OpenBao deaktiviert die unauthentifizierten
# sys/generate-root/* Endpunkte seit 2.5.3. Ohne diese Pfade kann dieser
# Login keinen Ersatz-Root-Token erzeugen – und die Unseal-Keys allein auch nicht.
path "sys/generate-root-token/attempt" { capabilities = ["create", "read", "update", "delete",
    "sudo"] }
path "sys/generate-root-token/update" { capabilities = ["create", "update", "sudo"] }
path "sys/decode-token" { capabilities = ["create", "update"] }

path "auth/*" { capabilities = ["create", "read", "update", "delete",
    "list", "sudo"] }
path "identity/*" { capabilities = ["create", "read", "update", "delete",
    "list"] }
path "secret/*" { capabilities = ["create", "read", "update", "delete",
    "list"] }
path "database/*" { capabilities = ["create", "read", "update", "delete",
    "list"] }
EOF

/ $ bao auth enable userpass
/ $ bao write auth/userpass/users/admin \
    password="$(read -rs -p 'Passwort: ' p; echo "$p")" \
    token_policies=admin token_ttl=1h token_max_ttl=8h

```

Das Passwort gehört in den Passwort-Manager, **bevor** Schritt 5 kommt. Unter OpenBao ist es Recovery-Material, nicht Komfort: Seit 2.5.3 sind die unauthentifizierten `sys/generate-root/*`-Endpunkte standardmäßig aus, und `bao operator generate-root` nutzt die authentifizierten `sys/generate-root-token`-Endpunkte. Drei Unseal-Keys allein sind also **kein Weg zurück** – es braucht zusätzlich einen Login mit genau diesen Pfaden in der Policy.

4. Den Ersatzzugang beweisen – nicht überspringen

```
# -token-only: sonst überschreibt 'bao login' das laufende Root-Token
# in ~/.vault-token, BEVOR bewiesen ist, dass der neue Zugang funktioniert.
/ $ ADMIN_TOKEN="$(bao login -method=userpass -token-only username=admin)"

/ $ BAO_TOKEN="$ADMIN_TOKEN" bao token lookup | grep policies
/ $ BAO_TOKEN="$ADMIN_TOKEN" bao policy list

# Der Break-Glass-Pfad selbst: -init startet eine Root-Generierung,
# -cancel bricht sie ab. Das beweist die Fähigkeit, ohne einen Root-Token
# zu erzeugen, den man dann verwahren müsste.
/ $ BAO_TOKEN="$ADMIN_TOKEN" bao operator generate-root -init
/ $ BAO_TOKEN="$ADMIN_TOKEN" bao operator generate-root -cancel

/ $ unset ADMIN_TOKEN
```

Erst wenn admin in den Policies steht, `policy list` funktioniert **und** das `generate-root -init/-cancel`-Paar durchläuft, ist der Ersatzzugang bewiesen. Schlägt irgendetwas davon fehl: Schritt 5 **nicht** ausführen.

5. Root-Token widerrufen

```
/ $ bao token lookup | grep policies      # [root]
/ $ bao token revoke -self
/ $ bao token lookup                      # muss jetzt fehlschlagen
```

Ab hier ist userpass der Weg hinein. Break Glass, falls der Zugang verloren geht:

```
bao login -method=userpass username=admin
bao operator generate-root -init
bao operator generate-root      # 3x mit Unseal-Keys
```

6. Snapshots

Auf der VM: ein systemd-Timer, der täglich `bao operator raft snapshot save` ausführt, das Archiv prüft und auf ein NAS kopiert (N° 3 im Detail). Im Cluster ist das Äquivalent ein CronJob im Namespace `openbao`, der mit einem eigenen ServiceAccount, einer Kubernetes-Auth-Rolle und der Policy

```
path "sys/storage/raft/snapshot" { capabilities = ["read"] }
```

den Snapshot zieht und in ein Objekt-Storage (S3-kompatibel, z. B. MinIO oder das NAS) schreibt. Der Vorteil gegenüber der VM: Kein periodischer Token in einer Datei – der CronJob meldet sich bei jedem Lauf mit seinem ServiceAccount an.

Bis der CronJob existiert, geht ein Snapshot von Hand:

```
kubectl exec -n openbao openbao-0 -- \
  sh -c 'BAO_TOKEN=... bao operator raft snapshot save /tmp/bao.snap' && \
  kubectl cp openbao/openbao-0:/tmp/bao.snap ./openbao-$(date +%Y%m%dT%H%M).snap
```

Und die Prüfung, die OpenBao selbst nicht anbietet (raft snapshot inspect gibt es nur bei Vault):
Das Archiv ist ein gzipped tar mit meta.json, state.bin und SHA256SUMS –

```
tar -tzf openbao-*.snap
tar -xz0f openbao-*.snap SHA256SUMS
tar -xz0f openbao-*.snap state.bin | sha256sum # muss zur Zeile oben passen
```

Einen Restore einmal proben, bevor man sich darauf verlässt. Ein ungetesteter Restore ist eine Annahme, kein Backup. Der Ablauf – auf einen frischen Cluster, mit –force und zwei verschiedenen Key-Sätzen nacheinander – steht in N° 3 und gilt im Cluster unverändert.

7. Konfiguration nach OpenTofu

Alles aus Teil III und diesem Teil ist Zustand, der bei einem Neuaufbau verloren ist. Die Objekte und ihre Ressourcen:

Objekt	Tofu-Ressource
Audit-Device	vault_audit
Policy admin, eso-demo	vault_policy
userpass + Admin-User	vault_auth_backend, vault_userpass_auth_backend_user (mit password_wo, damit das Passwort nie im State landet)
Kubernetes-Auth	vault_auth_backend (type = "kubernetes"), vault_kubernetes_auth_backend_config, vault_kubernetes_auth_backend_role

Der Provider heißt hashicorp/vault – einen openbao/openbao-Provider gibt es nicht. OpenBao hat die Vault-HTTP-API behalten, der Provider funktioniert unverändert und wird über VAULT_ADDR auf OpenBao gezeigt. Wie das Verzeichnis aussieht, inklusive State-Verschlüsselung und dem Umgang mit dem Admin-Passwort, ist in N° 3 beschrieben und lässt sich eins zu eins übernehmen.

Teil VII – Was schiefgehen kann

Pod bleibt 0/1 Running

Erwartet, solange OpenBao versiegelt oder nicht initialisiert ist. `kubectl exec ... bao status` zeigt, welcher Fall vorliegt. Erst wenn `Sealed false` und der Pod trotzdem nicht Ready wird, ist etwas kaputt – dann die Probe im StatefulSet ansehen (`kubectl describe pod`).

Log voller `service_registration ... 403`

Alle fünf Sekunden eine Warnung: `service_registration "kubernetes"` ist konfiguriert, aber der ServiceAccount darf den eigenen Pod nicht patchen. Im Standalone-Modus rendert das Chart die nötige Role nicht. Den Block aus der Server-Config entfernen (siehe Teil I) oder eine Role mit `get`, `update`, `patch` auf pods und ein RoleBinding auf den ServiceAccount `openbao` anlegen.

`helm upgrade` ändert nichts

`OnDelete`. Pod löschen, `unseal`. Siehe Teil I.

`403 permission denied` bei allem

Kein Token, abgelaufenes Token, oder Token vom falschen System. `bao token lookup` zuerst. Erst wenn das funktioniert und ein bestimmter Pfad trotzdem 403 gibt, ist es die Policy – `bao token capabilities <pfad>` zeigt deny.

Kubernetes-Login schlägt fehl

Meldung	Ursache
<code>service account name not authorized</code>	<code>bound_service_account_names</code> passt nicht
<code>namespace not authorized</code>	<code>bound_service_account_namespaces</code> passt nicht
Post "https://kubernetes.default.svc:443/.../tokenreviews": ...	OpenBao erreicht den API-Server nicht, oder sein eigenes Token darf keinen TokenReview stellen
<code>permission denied</code> erst beim Lesen eines Pfads	Login war erfolgreich; die Policy der Rolle erlaubt den Pfad nicht

OpenBao nimmt keine Anfragen mehr an, Log sagt audit

Das Audit-Volume ist voll oder nicht beschreibbar. OpenBao verweigert dann absichtlich. Volume vergrößern oder Log rotieren – und danach einen zweiten Audit-Device einrichten, damit das nicht wieder passiert.

Nach Node-Wartung: versiegelt

Erwartet. `kubect1 exec -it -n openbao openbao-0 -- bao operator unseal`, dreimal. Wenn das zu oft passiert: Auto-Unseal, siehe Teil II.

Anhang A – Alle Kommandos

```
# — Helm —————
helm repo add openbao https://openbao.github.io/openbao-helm
helm upgrade --install openbao openbao/openbao --version 0.29.4 \
  -n openbao --create-namespace -f k8s/values.yaml

# — init / unseal (Shell im Pod, normales Terminal) —————
kubectl exec -it -n openbao openbao-0 -- sh
bao operator init -key-shares=5 -key-threshold=3      # Ausgabe -> Passwort-Manager
bao operator unseal                                  # x3
bao status
read -rs BAO_TOKEN; export BAO_TOKEN                 # kein 'bao login' im Pod

# — Kubernetes-Auth —————
bao auth enable kubernetes
bao write auth/kubernetes/config kubernetes_host="https://kubernetes.default.svc:443"
bao write auth/kubernetes/role/eso-demo \
  bound_service_account_names=demo bound_service_account_namespaces=demo \
  token_policies=eso-demo token_ttl=1h

# — Härtung (Reihenfolge einhalten) —————
bao audit enable file file_path=/openbao/audit/audit.log
bao policy write admin - < admin.hcl
bao auth enable userpass
bao write auth/userpass/users/admin password=... token_policies=admin token_ttl=1h token_max
  _ttl=8h
ADMIN_TOKEN="$(bao login -method=userpass -token-only username=admin)"
BAO_TOKEN="$ADMIN_TOKEN" bao operator generate-root -init
BAO_TOKEN="$ADMIN_TOKEN" bao operator generate-root -cancel
bao token revoke -self                               # erst nach bestandenem Test

# — Snapshot von Hand —————
kubectl exec -n openbao openbao-0 -- sh -c 'bao operator raft snapshot save /tmp/bao.snap'
kubectl cp openbao/openbao-0:/tmp/bao.snap ./openbao-$(date +%Y%m%dT%H%M).snap
tar -xz0f openbao-*.snap state.bin | sha256sum
```

Anhang B – Glossar

Seal / Unseal – OpenBao startet mit verschlüsseltem Storage und ohne den Schlüssel dazu. Unseal setzt den Master-Key aus Shamir-Anteilen zusammen; bis dahin beantwortet OpenBao nur status.

Shamir 5/3 – Der Master-Key ist in fünf Anteile zerlegt, drei beliebige davon rekonstruieren ihn.

Raft – Integriertes Storage-Backend mit Konsens-Protokoll. Auch mit einem Knoten sinnvoll, weil nur Raft Snapshots kann.

Root-Token – Der Token aus init. Unbegrenzt gültig, nicht einschränkbar. Wird nach dem Bootstrap widerrufen und bei Bedarf per generate-root neu erzeugt.

Auth-Methode – Wie ein Client seine Identität nachweist. kubernetes prüft ServiceAccount-Tokens per TokenReview; userpass prüft Passwörter.

Rolle (Auth) – Bindet Identitätsmerkmale (ServiceAccount, Namespace) an Policies und Token-Eigenschaften.

Policy – Erlaubte Pfade und Capabilities. Default: alles verboten.

Audit-Device – Protokolliert jede Anfrage und Antwort (mit gehashten Secrets). Kann OpenBao nicht schreiben, verweigert es Anfragen.

TokenReview – Kubernetes-API, mit der ein Dritter prüfen lässt, ob ein ServiceAccount-Token gültig ist und zu wem es gehört.

Break Glass – Der Weg zu einem neuen Root-Token, wenn der alte weg ist. Unter OpenBao braucht er Unseal-Keys **und** einen Login mit sys/generate-root-token/*.

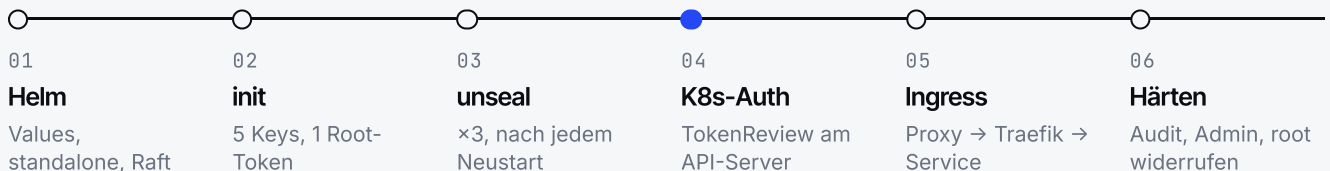
OnDelete – Update-Strategie des StatefulSets: neue Konfiguration wird erst wirksam, wenn der Pod von Hand gelöscht wird.

Der Pod läuft. Er ist versiegelt. Und genau das ist richtig.

Dieser Leitfaden dokumentiert den Aufbau eines OpenBao im Kubernetes-Cluster, so wie er tatsächlich passiert ist: Helm-Values mit Begründung, init und unseal in einer Pod-Shell, Kubernetes-Auth mit TokenReview, der Weg von außen durch Proxy und Traefik.

Und den Zustand danach, ehrlich: kein Audit-Device, der Root-Token als Datei im Pod, ein service_registration-Block, der 22 Stunden lang alle fünf Sekunden mit 403 scheiterte. Dazu die Härtung, die daraus folgt – in der Reihenfolge, die einen Lockout ausschließt.

```
-- Die eine Zeile, die Backups überhaupt möglich macht
storage "raft" {
  path = "/openbao/data"
}
```



kubernetes	RKE2 v1.36 · Traefik · Longhorn
openbao	2.6.2 · chart 0.29.4 · raft
helm	3.16 · standalone
auth	kubernetes · TokenReview
storage	10Gi PVC · Retain

24 Seiten · 7 Teile · 2 Anhänge
Vertiefung: **Vault in Practice**
leanpub.com/vault-in-practice
Gesetzt in Inter Tight, Inter und JetBrains Mono.
Alle Ausgaben sind echte Terminal-Ausgaben.